PRODUCT REQUIREMENTS DOCUMENT (PRD)

Sistem Terintegrasi Activity Planner, Budget Planning & Document Generator

Spesifikasi Fungsional, Standar UI/UX Minimalis & Arsitektur Database Skala Besar

Versi Dokumen: 1.0 (Initial Scope)
Status: Draft Review
Platform: Web App (Desktop & Mobile)

Atribut Dokumen	Detail Spesifikasi	Fokus Engineering	Target Kualitas
Ruang Lingkup	Kegiatan, Anggaran (RAB), Surat	**Performa Database**	Siap skala jutaan–ratusan juta baris
Format Output Surat	Native Microsoft Word (`.docx`)	**Standar UI/UX**	Clean, Minimalist, Anti-"AI Slop"
Prinsip Kerja	Zero Assumption, Second-Check	**Interaksi Form**	Smart Autofill, Debounced Search, Toast

1. Tujuan Produk & Prinsip Kerja (Core Principles)

Aplikasi web ini dirancang untuk mengelola perencanaan kegiatan (**Activity Planner**), perencanaan anggaran (**Budget Planning**), dan pembuatan dokumen/surat resmi (**Document Generator**) dalam satu alur data yang saling berkesinambungan. Data yang diinput pada satu modul otomatis dapat digunakan kembali (*autofill/linked*) pada modul lainnya tanpa duplikasi input manual.

1. Zero Assumption Policy
Sistem hanya membangun fitur berdasarkan spesifikasi yang disepakati. Titik yang belum memiliki kepastian aturan bisnis dipisahkan dalam daftar konfirmasi resmi.

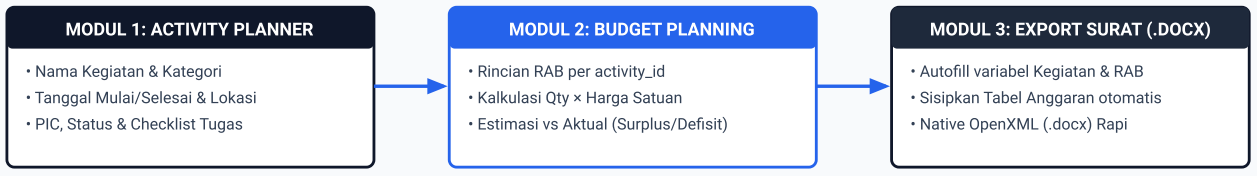
2. No Over-Engineering
Copywriting UI, struktur komponen, dan alur logika dibuat lugas. Tidak ada abstraksi berbelit atau teks panjang untuk fungsi yang sederhana.

3. No Random Changes & Second-Check
Setiap perubahan skema atau UI memiliki alasan fungsional yang jelas dan melewati tahap verifikasi ulang (*second-check*) sebelum diimplementasikan.

4. Anti-"AI Slop" UI Design
Menghindari gradien warna mencolok (ungu/neon khas template AI) dan dekorasi tanpa fungsi. Fokus pada tipografi bersih, *whitespace* terukur, dan hierarki informasi.

2. Arsitektur & Alur Kesinambungan Data (Core Workflow)

Ketiga modul utama saling berelasi dengan **Kegiatan** (`Activity`) sebagai entitas induk (namun tetap fleksibel apabila pengguna ingin membuat surat atau anggaran umum secara mandiri):



3. Spesifikasi Fungsional (Functional Requirements)

3.1. Modul Activity Planner (Perencanaan Kegiatan)

- **Manajemen Kegiatan (CRUD):** Membuat, mengubah, melihat, dan mengarsipkan kegiatan yang mencakup Judul Kegiatan, Kategori, Tanggal Mulai & Selesai, Lokasi, Penanggung Jawab (PIC), serta Status (*Draft, Terjadwal, Berjalan, Selesai, Dibatalkan*).
- **Tampilan Kalender & Daftar (List/Table View):** Dilengkapi filter cepat berdasarkan rentang tanggal, status pelaksanaan, dan ringkasan ketersediaan anggaran.
- **Sub-item / Checklist Persiapan:** Daftar tugas singkat di dalam setiap kegiatan untuk melacak kesiapan operasional sebelum hari pelaksanaan.
- **Quick Action Link:** Tombol pintas langsung dari halaman detail kegiatan untuk *"Buat Rencana Budget"* atau *"Buat Surat Terkait"* dengan data kegiatan yang langsung terikat.

3.2. Modul Budget Planning (Perencanaan Anggaran)

- **Struktur Anggaran Terhubung:** Setiap rencana budget terikat langsung pada `activity_id` tertentu (atau kategori operasional umum apabila berdiri sendiri).
- **Itemisasi Anggaran Presisi:** Mencakup pengelompokan Kategori Pengeluaran (misal: Konsumsi, Perlengkapan, Transportasi, Dokumentasi) dengan kolom `Nama Item`, `Kuantitas (Qty)`, `Satuan`, `Harga Satuan (Estimasi)`, `Total Estimasi` (kalkulasi otomatis `Qty × Harga Satuan`), serta `Realisasi/Aktual`.
- **Ringkasan Otomatis (Summary Bar):** Kalkulasi *real-time* untuk Total Pagu/Estimasi, Total Realisasi, dan Selisih (Surplus/Defisit) tanpa perlu memuat ulang (*reload*) halaman.
- **Reusable Budget Items (Autofill):** Saat pengguna mengetik nama item pengeluaran yang pernah dibuat sebelumnya, sistem otomatis menyarankan satuan dan harga satuan terakhir sebagai referensi cepat.

3.3. Modul Pembuatan Surat & Export Word (`.docx`)

- **Pemilihan Template Surat:** Mendukung struktur surat resmi standar meliputi Surat Undangan, Surat Permohonan Dana/Pencairan Anggaran, Surat Tugas, dan Surat Pemberitahuan.
- **Dynamic Variable & Autofill dari Kegiatan/Budget:** Saat pengguna memilih kegiatan terkait, *field* seperti nama kegiatan, hari/tanggal, waktu, lokasi, dan total anggaran otomatis terisi ke dalam form surat. Tersedia opsi satu klik untuk menyisipkan tabel ringkasan anggaran langsung ke dalam lampiran surat.
- **Live Preview (Kertas A4/F4):** Pratinjau visual di layar web yang merepresentasikan proporsi halaman cetak sebelum dokumen diunduh.

> **Standar Engineering: Native .docx Export Engine (Bukan Konversi HTML-ke-Word)**
> Mengubah HTML biasa menjadi `.doc` sering merusak margin, tabulasi titik dua (`:`), dan batas tabel saat dibuka di Microsoft Word. Sistem ini wajib menggunakan generator **OpenXML murni** (seperti `docx` atau `docxtemplater`) dengan aturan ketat: (1) Ukuran kertas A4/F4 dan margin terkunci presisi dalam satuan *twips/millimeters*; (2) Kop surat dan atribut `Nomor :`, `Lampiran :`, `Perihal :` disusun menggunakan *borderless table grid* agar titik dua sejajar sempurna di semua versi MS Word; (3) Area tanda tangan menggunakan *invisible grid* agar tidak bergeser saat diedit manual; serta (4) Tabel RAB memiliki *header row repeat* otomatis jika memanjang lintas halaman.

4. Standar UI/UX & Micro-Interactions

Palet Warna Minimalis: `#FFFFFF` Surface · `#F8FAFC` Subtle BG · `#0F172A` Text Utama · `#2563EB` Primary Accent · `#16A3`

Komponen Pengalaman	Standar Implementasi & Detail Teknis
Tipografi & Angka Tabel	Menggunakan *sans-serif* bersih (`Inter` / `Plus Jakarta Sans`) untuk UI aplikasi, dan font standar surat (`Times New Roman` / `Arial`) pada pratinjau dokumen. Seluruh nominal anggaran wajib menerapkan `font-variant-numeric: tabular-nums` agar digit ribuan sejajar lurus secara vertikal.

5. Desain Database & Skalabilitas (Skala Jutaan–Ratusan Juta Baris)

Untuk memastikan database tetap hemat *resource* (penggunaan CPU & RAM rendah serta waktu eksekusi *query* di bawah `20ms` meskipun tabel mencapai puluhan hingga ratusan juta baris), arsitektur sistem menggunakan **PostgreSQL** dengan standar industri berikut:

5.1. Prinsip Skema Inti & Efisiensi Memori

- **Primary Key Sekuensial (`BIGINT` atau `UUIDv7`):** Menghindari penggunaan `UUIDv4` (random) karena menyebabkan *B-Tree index fragmentation* parah dan membengkakkan konsumsi RAM (*buffer pool*) saat tabel menembus jutaan baris. Penggunaan `BIGINT` (8 bytes) atau `UUIDv7` (*time-ordered*) memastikan penulisan *index* selalu berurutan (*append-only*).
- **Tipe Data Keuangan Presisi Tinggi:** Seluruh kolom nominal anggaran wajib menggunakan `NUMERIC(15, 2)` atau `BIGINT`, dan **dilarang keras** menggunakan `FLOAT` atau `DOUBLE` untuk mencegah kesalahan pembulatan desimal (*floating-point rounding error*).
- **Denormalisasi Terkontrol untuk Summary:** Menyimpan nilai agregat `total_estimated_budget` dan `total_actual_budget` langsung pada tabel `activities` yang diperbarui secara transaksional. Strategi ini membuat pembacaan daftar ribuan kegiatan beserta total anggarannya tidak memerlukan `JOIN` dan `SUM()` berat ke ratusan juta baris item anggaran.

5.2. Struktur Tabel Utama (PostgreSQL DDL)

```
      -- 1. Tabel Kegiatan (Activities) - Menyimpan summary budget ter-denormalisasi CREATE
TABLE activities ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, user_id BIGINT NOT NULL,
title VARCHAR(255) NOT NULL, location VARCHAR(255), pic_name VARCHAR(150), status SMALLINT NOT
NULL DEFAULT 1, -- 1:Draft, 2:Scheduled, 3:Ongoing, 4:Completed, 5:Cancelled start_date
TIMESTAMPTZ NOT NULL, end_date TIMESTAMPTZ, total_estimated_budget NUMERIC(15, 2) NOT NULL
DEFAULT 0, total_actual_budget NUMERIC(15, 2) NOT NULL DEFAULT 0, created_at TIMESTAMPTZ NOT
NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), deleted_at TIMESTAMPTZ ); --
2. Tabel Item Anggaran (Budget Items) - Potensi pertumbuhan baris terbesar CREATE TABLE
budget_items ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, activity_id BIGINT NOT NULL
REFERENCES activities(id), category VARCHAR(100) NOT NULL, item_name VARCHAR(255) NOT NULL, qty
NUMERIC(10, 2) NOT NULL DEFAULT 1, unit VARCHAR(50) NOT NULL, unit_price NUMERIC(15, 2) NOT NULL
DEFAULT 0, estimated_subtotal NUMERIC(15, 2) GENERATED ALWAYS AS (qty * unit_price) STORED,
actual_subtotal NUMERIC(15, 2) NOT NULL DEFAULT 0, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
); -- 3. Tabel Dokumen / Surat (Documents) & 4. Kamus Autofill Cepat (Autofill Dictionary)
CREATE TABLE documents ( id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, activity_id BIGINT
REFERENCES activities(id), user_id BIGINT NOT NULL, doc_number VARCHAR(100) NOT NULL, doc_type
SMALLINT NOT NULL, subject VARCHAR(255) NOT NULL, recipient_name VARCHAR(255) NOT NULL,
content_payload JSONB NOT NULL, issued_date DATE NOT NULL, created_at TIMESTAMPTZ NOT NULL
DEFAULT NOW(), deleted_at TIMESTAMPTZ ); CREATE TABLE autofill_dictionary ( id BIGINT GENERATED
ALWAYS AS IDENTITY PRIMARY KEY, user_id BIGINT NOT NULL, field_type VARCHAR(50) NOT NULL, --
'location', 'budget_item', 'recipient', 'unit' value_text VARCHAR(255) NOT NULL, metadata JSONB,
-- Contoh: { "last_unit": "Box", "last_price": 25000 } usage_count INT NOT NULL DEFAULT 1,
last_used_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), UNIQUE (user_id, field_type, value_text) );
```

5.3. Strategi Indexing, Cursor Pagination & Partisi

Untuk menjaga kecepatan *query* tetap konstan seiring bertambahnya data: (1) Wajib menggunakan **Keyset / Cursor-Based Pagination** (`WHERE user_id = $1 AND (start_date, id) < ($last_date, $last_id) ORDER BY start_date DESC, id DESC LIMIT 20`) dan melarang penggunaan `OFFSET` besar; (2) Menerapkan **Partial Index** (`WHERE deleted_at IS NULL`) serta **GIN Trigram Index** (`pg_trgm`) pada tabel `autofill_dictionary` agar pencarian *autocomplete* tetap instan; serta (3) Menyiapkan desain **Table Partitioning** berdasarkan rentang waktu pada `budget_items` jika melampaui 50 juta baris.

6. Rekomendasi Tech Stack (Pragmatis & Standar Industri)

Pemilihan teknologi difokuskan pada stabilitas jangka panjang, dukungan tipe data yang ketat (*type-safety*), performa *rendering* yang ringan, serta kemampuan menghasilkan dokumen `.docx` dengan tingkat presisi tinggi:

Layer Arsitektur	Teknologi Utama	Alasan Pemilihan & Keunggulan Teknis
Fullstack Framework	Next.js (App Router) + TypeScript	Mendukung *Server Components*, *code splitting* otomatis per halaman, dan keamanan tipe data penuh dari database hingga ke komponen UI.
Styling & UI Primitives	Tailwind CSS + Radix UI / shadcn	Bebas *runtime CSS overhead*, aksesibilitas keyboard bawaan untuk *autocomplete*/dialog, serta dikustomisasi ketat agar bersih dari kesan template AI.
State & Data Fetching	TanStack Query (React Query)	Mengelola *caching* pintar, *optimistic UI updates* saat mengubah status/anggaran, serta sinkronisasi *debounced search autofill*.
Form & Schema Validation	React Hook Form + Zod	Re-render minimal saat pengguna mengetik banyak baris anggaran, dengan satu skema validasi yang sama untuk sisi klien dan server.
Database & ORM Layer	PostgreSQL + Drizzle ORM + PgBouncer	Drizzle menghasilkan SQL yang sangat efisien tanpa *overhead* berat, mendukung *cursor pagination*, dan PgBouncer menjaga koneksi tetap hemat RAM.
Word (`.docx`) Engine	`docx` (OpenXML) / `docxtemplater`	Menyusun struktur XML Microsoft Word secara native (margin milimeter, tabel *borderless* rapi, kop surat presisi) tanpa cacat konversi HTML.

7. Second-Check & Daftar Konfirmasi (Zero Assumption)

Sesuai aturan kerja untuk **tidak membuat asumsi**, **tidak melakukan perubahan acak**, dan selalu melakukan *second-check*, seluruh spesifikasi di dalam PRD ini telah diverifikasi agar hanya berfokus pada *Activity Planner*, *Budget Planning*, dan *Document Generator (`.docx`)*. Sebelum melangkah ke tahap penulisan kode dan skema final, mohon konfirmasi 4 poin berikut:

Konfirmasi 1 **Format & Struktur Surat (.docx)**
Apakah Anda sudah memiliki contoh file surat (`.docx`) dengan posisi kop dan penomoran baku yang biasa dipakai, atau kita mulai dengan 2 template standar (*Surat Undangan Kegiatan* & *Surat Permohonan Anggaran*)?

Konfirmasi 2 **Skala Akses Pengguna**
Untuk fase pertama ini, apakah aplikasi hanya digunakan oleh Anda sendiri (*Single-User / Personal Admin*), atau membutuhkan sistem *Multi-User* (beberapa anggota tim dapat login dan berkolaborasi)?

Konfirmasi 3 **Cakupan Alur Anggaran**
Apakah modul budget pada tahap awal ini cukup mencatat **Pengeluaran (Estimasi vs Realisasi)** per kegiatan, atau perlu mencatat **Sumber Pemasukan/Dana** untuk kegiatan tersebut juga?

Konfirmasi 4 **Persetujuan Tech Stack**
Apakah Anda setuju menggunakan kombinasi **Next.js + TypeScript + Tailwind CSS + PostgreSQL** seperti yang direkomendasikan di atas, atau ada preferensi teknologi lain?

Langkah Selanjutnya Setelah Konfirmasi (Next Action Plan)

Setelah 4 poin di atas dikonfirmasi, kita akan langsung masuk ke **Tahap 1: Finalisasi Database Migration & Struktur Folder Project**, dilanjutkan dengan **Tahap 2: Pembuatan UI Activity & Budget Planner**, serta **Tahap 3: Pembuatan Engine Export Word (`.docx`)**.